

Solving the multi-objective bike routing problem by
meta-heuristic algorithmsPedro Nunes^{a,*} , Ana Moura^b  and José Santos^a ^a*Department of Mechanical Engineering/TEMA, University of Aveiro, Campus Universitário Santiago, Aveiro 3810-193, Portugal*^b*Department of Economics, Management, Industrial Engineering and Tourism/GOVCOPP, University of Aveiro, Campus Universitário Santiago, Aveiro 3810-193, Portugal**E-mail: pnunes@ua.pt [Nunes]; ana.moura@ua.pt [Moura]; jps@ua.pt [Santos]*

Received 17 March 2021; received in revised form 18 September 2021; accepted 27 December 2021

Abstract

The Multi-Objective Bike Routing Problem (MOBRP) addressed in this paper consists in finding a set of good solutions that represents the trade-off between cyclist preferences when choosing a route. We propose a new genetic approach to solve the MOBRP (NGA-MOBRP) with a new mutation operator. To test the approach, we consider four conflicting criteria: safety, travel distance, travel time, and comfort. The well-known Nondominated Sorting Genetic Algorithm II (NSGA-II) is also implemented in the MOBRP context, and its weaknesses are identified and mitigated in the proposed NGA-MOBRP. The developed approach is compared with other published works in this field, namely another genetic algorithm and a Multi-Objective Simulated Annealing (MOSA) approach. The computational results show that the approach developed in this work obtains better results than the previously mentioned published approaches.

Keywords: bike routing; genetic algorithms; meta-heuristics; multi-objective shortest path; mutation operator

1. Introduction

With the increasing demand for alternative and sustainable means of transport, the bicycle's popularity is growing at a considerable rate (ECF, 2019), because it is efficient, comfortable, and it may be faster than motorized vehicles for short distances in an urban environment (Koning and Kopp, 2014), especially if traffic congestion is high (Lindsay et al., 2011). The growing potential of this mean of transportation is enormous, especially, when we realize that almost half of the car trips in cities are less than 5 km (European Commission Mobility and Transport, 2019).

However, riding a bike represents specific challenges for the cyclist, because there are plenty of route alternatives and each of them may represent a different trade-off between cyclists preferences.

*Corresponding author.

© 2022 The Authors.

International Transactions in Operational Research © 2022 International Federation of Operational Research Societies
Published by John Wiley & Sons Ltd, 9600 Garsington Road, Oxford OX4 2DQ, UK and 350 Main St, Malden, MA02148, USA.

Moreover, bike routing is more complex than traditional motor vehicle routing. In the latter case, the main objective is typically to minimize parameters that are fairly simple to determine, such as travel time, distance, or fuel consumption (Schmitt and Baldo, 2018), while in bike routing, there are a large set of features that influence the cyclists' behavior (Broach et al., 2012). Some of these features are the existence of bike lanes and their conditions (Pucher and Buehler, 2008), slope, route singularities (Menghini et al., 2010), and the existence of physical separation from motorized traffic or obstacles (Dondi et al., 2011). The challenges for cyclists are even harder if they are beginners or have poor knowledge of the surrounding areas. For this reason, the existence of a practical decision tool is seen as a factor that promotes bicycle use (Akar and Clifton, 2009).

The **multi-objective bike routing Problem (MOBRP) is a multi-objective shortest path (MSP)** problem in the particular setting of bike-related criteria. This problem is of utmost importance, to create real-time tools for cyclists. Since these applications should solve the problem in a timely manner in a real-road network, the computational performance should not vary considerably in each instance. Moreover, for a real cyclist, it is not mandatory to have the exact solution for the problem. The most important is to have a set of diverse solutions that satisfy the different criteria. Thus, meta-heuristic approaches are appealing, because they are able to achieve several solutions, all near the optimal Pareto set (approximated Pareto set) in a reasonable computational time. One of **the most popular meta-heuristics for multi-objective problems is the Nondominated Sorting Genetic Algorithm II (NSGA-II) proposed by Deb et al. (2000), however, when applied to the MOBRP, this approach revealed some limitations,** which are mitigated by the new genetic approach (NGA-MOBRP) proposed in this work. The computational experiments compare these approaches with the works proposed by Nunes et al. (2020a, 2020b).

The main contributions of this work are:

- Development of a new genetic approach that mitigates the NSGA-II limitations in the specific scope of MOBRP, with a new selection mechanism and with a semi-controlled population size;
- Development of new genetic operators in the context of the MOBRP, namely a mutation operator and a mechanism to generate the initial population;
- Implementation of the NSGA-II to solve the MOBRP and identification of its limitations;
- Extension of the approaches developed by Nunes et al. (2020a, 2020b), by making new computation tests considering new instances and new conflicting objectives;
- Comparative study between four different meta-heuristic approaches.

This paper is organized as follows: some related work is presented in the next section (Section 2). A mathematical formalization of the MOBRP is presented in Section 3. A detailed description of the proposed approach is given in Section 4. The computational experiments are presented in Section 5, while Section 6 discusses the obtained results. Finally, in Section 7 some conclusions are highlighted.

2. Related work

There are three different types of approaches to solve multi-objective problems: *a priori*, *interactive*, and *a posteriori*. In the first approach, the decision maker provides preferences for the different

objectives; in *interactive* approaches, the preference decisions are made during the solving process; while in *a posteriori* approaches, a set of potential nondominated solutions is generated, to be chosen by the decision maker (Jozefowicz et al., 2008).

An example of an *a priori* approach to MOBRP is to weight each objective function, in order to **create a unified cost function**, which reduces the MOBRP to the shortest path problem (SPP). Hochmair and Fu (2009) used this approach to create a web-based bicycle trip planning using the ArcGIS Server framework and some Google APIs. According to this work, the user must choose previously one of the five available criteria. Then, the shortest path is calculated and displayed in the framework. Luxen et al. (2011), employed a similar methodology, by using contraction hierarchies together with OpenStreetMap data to find the shortest path in a graph. On the other hand, Hrnčir et al. (2014) used a cost vector for each criterion (travel time, comfort, quietness, and flatness), and created different user profiles, by giving different weights to each criterion. A single solution was then calculated using the A* algorithm for the chosen profile.

Despite *a priori* approaches being practical and computationally light, the application of such methods results in a single final solution. However, to have a set of good solutions may be essential, especially in problems where the most suitable solution strongly depends on the personal preferences, such as the MOBRP. Moreover, to employ *a priori* or *interactive* approaches requires the inputs of the decision maker, in the beginning of the process or during the determination of the solutions set, which is not always possible. For this reason, most of the research efforts have been directed to the *a posteriori* approaches. The first *a posteriori* approach applied to a multi-objective routing problem was formalized by Martins (1984) who proposed multiple objective functions to describe the problem and created the multi-criteria label-setting algorithm to solve it. A multi-label correcting algorithm was used by Song et al. (2014) to select a set of Pareto routes. Together with the previous algorithm, the authors also applied a route selection algorithm based on hierarchical clustering to reduce the size of the Pareto set. Paixão and Santos (2013) present extensive computational tests for MSP problems in randomly generated graphs. Despite the improvements to the existing label-setting algorithms, the computational times for these approaches are not suitable for real-time use, or they have not been tested in real networks. Feasible approaches in real networks have been proposed by Hrnčir et al. (2015), who developed speedups for bike routing algorithms that were applied in the heuristic-enabled Dijkstra algorithm developed by Hrnčir et al. (2017).

Evolutionary approaches also have been used in routing problems. Genetic algorithms have been used by Arunadevi et al. (2007), who implemented a parallel genetic algorithm (PGA) using High-performance Cluster (HPC) and by Kang and Fricker (2018), who created a practical procedure to compute the cost function of each edge in a network for cyclists, incorporating distance and perceived risk. Bahmankhah et al. (2019) implemented the NSGA-II to solve a three-criteria problem for short distance trips, while Nunes et al. (2020b) proposed a genetic approach to solve the bi-criteria bike routing problem. Other meta-heuristics were implemented in routing context by Osaba et al. (2018) to balance the trade-off between the length of the route and its safety level, and by Chang et al. (2005) to find a set of nondominated paths in a dynamic network. Nunes et al. (2020a) implemented a Multi-Objective Simulated Annealing (MOSA) approach to solve the MOBRP, based on the works developed by Suppapitnarm et al. (2000) and Sankararao and Yoo (2011). Despite being a common method employed to solve multi-objective problems, evolutionary approaches, such as genetic algorithms represent some challenges

in MSP problems, because operators such as crossover and mutation should respect the nodes' connectivity in the graph. For this reason, few approaches employ this method to perform the search for feasible paths. For example, Bahmankhah et al. (2019) employed the NSGA-II to find new numerical solutions for the considered cost functions, however, the feasible paths were proposed, based on the authors' simulation study. Li et al. (2013) proposed operators for the NSGA-II algorithm in the context of MSP problem, however, the generation of the first population, as well as some other operators, have inefficiencies that are not suitable for real-time tools.

The bi-objective shortest path (BSP) problem can be considered a variant of the MOBSP, which takes into account two conflicting objectives. In this field, many strategies and works have been proposed (Raith and Ehrgott, 2009), for example, Ehrgott et al. (2012) proposed a model to determine the route choice among commuter cyclists, by considering the trade-off between travel time and route suitability, and implemented a label-based algorithm to solve it. Pacheco et al. (2013) applied a tabu search meta-heuristic to solve the bi-objective bus routing, while, Machuca and Mandow (2012), and Duque et al. (2015) proposed efficient and exact algorithms to solve the BSP problem. Other works, which deal with different bi-objective routing problems, have been proposed: Demir et al. (2014) presented an adaptive large neighborhood search algorithm to minimize fuel consumption and driving time of vehicles to serve a set of costumers; Martínez-Salazar et al. (2014) proposed a new representation for the transportation location routing problem and applied two meta-heuristics to solve it; and Vidal et al. (2014) developed a framework and a genetic-based meta-heuristic to solve different variations of routing problems.

Despite the developments in the field of MOBSP, few approaches consider more than two criteria. Some approaches show potential, namely, the work presented by Hrnčir et al. (2017), which demonstrates that it is feasible to find nondominated routes with good computational performance, and the works developed by Ehrgott et al. (2012) and by Duque et al. (2015) that also had good performances, especially the latter one. However, these works only consider two criteria.

The MOBSP is a particular case of multi-objective problems. Meta-heuristic approaches are widely used in the literature to solve these problems, however, in the specific case of the MOBSP, the use of these techniques is not as developed as it is in other multi-objective problems. Moreover, existing works that demonstrate good performance does not consider more than three conflicting objectives, or were not applied to real problems. This fact motivated the development of operators to apply of meta-heuristics, such as the NSGA-II, to solve this problem. However, the tests made revealed some limitations of this approach, which motivated the development of the NGA-MOBSP to mitigate these issues.

The proposed approach is tested with two distinct real cycling networks, and compared with three other approaches, in terms of computational cost and the quality of the final set of solutions. The approaches tested are: the NSGA-II, a genetic-based approach, whose mutation operator is inspired in the work presented by Nunes et al. (2020a); the genetic algorithm proposed by Nunes et al. (2020b), that is a genetic approach in which a mutation operator was not applied; and the MOSA proposed by Nunes et al. (2020a), which is a different meta-heuristic applied to the MOBSP. Note that the proposed approach differs from the NSGA-II, since a new mechanism to select individuals and a semi-controlled population size schema during the creation of new populations, is applied. Moreover, to apply these algorithms, a mutation operator for the MOBSP, inspired in the work proposed by Nunes et al. (2020a), is proposed.

3. Problem formulation

3.1. Cycling network

Let us represent the cycling network by a directed graph $G(V, E, \vec{c})$, where V is the set of nodes, E is the set of edges, and $\vec{c}$ is a cost vector associated to each edge. The nodes represent junctions or places with any relevant feature, such as traffic lights, crosswalks, walkways, bumps, among others.

$E \subseteq \{(u, v) | (u, v \in V) \wedge u \neq v\}$ represents the edges. If the route is a one-way route, there is only one edge connecting the two nodes, otherwise, there are two edges, one for each direction.

$\vec{c}$ is a vector of nonnegative costs that represents the cost of transverse each edge (u, v) , according to each criterion, $\vec{c} = (c_1, c_2, \dots, c_k)$.

A path P is a sequence of edges, which joins a sequence of nodes to be visited, in order to connect origin and destination nodes, $P = \{n_0, n_1, n_2, \dots, n_d\}$, where n_0 represents the source node, and n_d the destination node.

3.2. Criteria and cost functions

Each edge has associated a k -dimensional cost vector, being k the number of considered criteria. In this work, we used $k = 4$, namely, travel distance, travel time, safety, and comfort. Each value of $\vec{c} = (c_1, c_2, \dots, c_k)$ is computed by a different cost function, of which formalization was inspired by the work presented by Hrnčir et al. (2014). However, some parameters were changed or added, based on what was considered a reasonable cycling behavior, according to recent works developed by Buehler and Dill (2016) and Vedel et al. (2017). Note that safety and comfort are modeled as perceived distances, which is a practical approach, and enables the development of simple heuristics to speed up the A* algorithm (as shown in Section 4.2). In the subsections below, each criterion is described in detail.

3.2.1. Travel distance

This criterion is computed using the haversine formula (Equation 1), which uses the nodes' geographical coordinates, latitude, and longitude, to calculate the distance between them.

$$L = 2R \times \sqrt{\alpha^2 + \cos(\text{lat}(u)) \times \cos(\text{lat}(v)) \times \theta^2}. \quad (1)$$

In Equation (1), u and v denote the nodes of an edge, L is the edge's length, α and θ are given by Equations (2) and (3), respectively. R is the Earth's radius (6371 Km) and lat and lon are the node's latitude and longitude. Despite being an approximation, this approach is a very fast method to calculate distances and its results are in accordance with distances provided by Google Maps.

$$\alpha = \frac{\sin(\text{lat}(u)) - \text{lat}(v)}{2}, \quad (2)$$

$$\theta = \frac{\sin(\text{lon}(u)) - \text{lon}(v)}{2}. \quad (3)$$

3.2.2. Travel time

In bike-routing, travel time depends on distance and other important parameters, such as road slope, surface conditions, and existence of road singularities, such as crossings, traffic signals, roundabouts, among others. Keeping this in consideration, we define travel time (in seconds) according to Equation (4).

$$T_{time} = \frac{L}{V(sl) \times ss} + F_s, \quad (4)$$

where L is the edge length in meters, $V(sl)$ is an estimated speed (m/s), that depends on the road slope (sl), according to Equation (5) (Romero et al., 2015), which reflects the cyclist's speed behavior according to the slope variation. Surface slowdown (ss) is a parameter that estimates the influence of surface conditions on a cyclist's speed, and, finally, the slowdown feature (F_s) that represents delays (in seconds) caused by certain road features, such as steps, traffic signals, or crossings.

$$V(sl) = \begin{cases} 7.582 \times e^{0.1072 \times sl}, & \text{if } sl < -0.92 \\ 5.787 \times e^{-0.1880 \times sl}, & \text{if } -0.92 < sl \leq 6 \\ 0.833, & \text{if } 6 < sl \leq 10 \\ 0, & \text{if } 10 < sl. \end{cases} \quad (5)$$

Although the edge length is calculated according to the haversine formula, the slope (sl), in percentage (%), is calculated by Equation (6).

$$sl(u, v) = 100 \times \frac{\Delta h}{L}. \quad (6)$$

Note that for this calculation, two nodes, u and v are required, thus the above equation is a trigonometric relation, where Δh is a difference between the altitude of each node, that is, $\Delta h = h(u) - h(v)$ and L is the horizontal length. Note that the nodes' order is important, since $sl(u, v) = -sl(v, u)$. According to the OpenStreetMaps (OSM) documentation page, nodes are also used to define the shape of the path, thus, is very unlikely to have significant slope variation in the same edge.

3.2.3. Safety

This criterion depends on many factors, such as slope, existence of road singularities, obstacles, and road type, since pedestrian paths, bike lanes, or primary roads have different safety costs. In general, cyclists prefer to travel longer distances, in order to ride in safer routes. Having this in consideration, the safety cost function is modeled as a perceived distance for the cyclist (in meters), given by Equation (7).

$$c_s = L \times Edge_{fs} \times c_{slope} + Node_{fs}, \quad (7)$$

where L is the edge's length, c_{slope} is a parameter that increases the safety cost of edges with sharp uphill or downhill. It is calculated by Equation (8).

$$c_{slope} = 2 - \frac{V(sl)}{V_{max}(sl)}, \quad (8)$$

where $V(sl)$ is the edge estimated speed and $V_{max}(sl)$ is the maximum estimated speed (24.74 km/h), which is reached for a negative slope of 0.92%, according to Romero et al. (2015). The $Edge_{fs}$ reflects the variation of safety perception, according to some edge features, such as type of highway and road singularities, while $Node_{fs}$ is a penalty, in meters, for the node features that decrease the safety perception.

3.2.4. Comfort

The comfort criterion is also subjective and depends on several parameters. In general, cyclists are willing to travel longer distances if there are better infrastructures for bicycles, better road conditions, good road surface, and flat roads. Thus, like safety, the comfort cost function is defined as a perceived distance (in meters), given by Equation (9).

$$c_c = L \times Edge_{fc} \times c_{slope} + Node_{fc}, \quad (9)$$

where L is the edge length (in meters), $Edge_{fc}$ reflects the variation of the perceived distance, according to some road features, such as surface type, existence of cycling infrastructures, junctions or other traffic singularities, while $Node_{fc}$ is a penalty estimation (in meters), for some node features that cause discomfort, such as steps, crossings, traffic signal, or speed bumps.

3.3. Data

The data needed to model the road network and instantiate the values of cost vectors, was obtained from two sources, OSM and Shuttle Radar Topography Mission (SRTM). OSM is a collaborative mapping project aimed at creating a free and editable map of the world. Maps are developed by local people and are reviewed by community members, by using GPS receivers, satellite photos, and other sources. One of the biggest advantages of this site is that all data are freely available and can be applied in any application.

The collected data are divided into two different types of elements, nodes and links. Each element may have additional information in a field designated by a tag, which gives important information about the element. This information identifies important road features that influence the cost functions, as can be seen in Tables 1 and 2. These values are a general insight to quantify how some road infrastructures affect the cost functions presented previously, since safety and comfort are very subjective concepts. Nevertheless, one can say that the values are realistic, since they result from the analysis of the works developed by Vedel et al. (2017) and Stinson and Bhat (2003), concerning the cyclist's preferences.

The SRTM is a space mission that aims to obtain a digital model of the Earth. It consists of an especially modified radar system to acquire the altimetry data, which is used to calculate the road slope. This data have a resolution of 1 arc second, or 30 m, and is available for almost worldwide since 2014.

3.4. Multi-objective dominance concepts

There are several possible paths that connect an origin node with a destination node. Each one of these paths, P , has a cost vector associated, $\vec{c} = (c_1, c_2, \dots, c_k)$. In this problem, the objective

Table 1
Node's features for each criterion

Key	Value	F (s)	Node_fc (m)	Node_fs (m)
Highway	Steps	20	300	–
	Give Way	2	100	100
	Crossing	4	100	120
	Traffic signals	2	100	70
	Elevator	15	150	–
Crossing	Stop	4	300	–
	Zebra	3	160	–
	Traffic signals	3	160	–
	Uncontrolled	3	160	120
Traffic calming	Bump	2	140	5
	Table	2	140	5

Table 2
Edge's features for each criterion

Key	Value	ss	Edge_fc	Edge_fs
Highway	Cicleway	–	0.57	0.15
	Steps	–	1.85	1.85
	Footway	–	–	0.81
	Pedestrian	–	–	0.81
	Track	–	–	0.81
	Path	–	–	0.81
	Primary	–	–	1.19
Surface	Ground	0.90	1.40	–
	Wood	0.80	2.00	–
	Sand	0.75	2.00	–
	Sett	0.85	1.65	–
	Gravel	0.75	1.70	–
	Paving Stones	0.85	1.50	–
	Unpaved	0.75	1.50	–
	Cobblestone	0.80	1.50	–
	Concrete	0.95	1.00	–
	Asphalt	1.00	1.00	–
Bicycle	Designated	–	0.57	0.43
	Yes	–	1.00	1.00
Junction	Roundabout	–	5.00	3.0
Bridge	Yes	–	7.00	1.19
	Movable	–	7.00	1.19
	Suspension	–	7.00	1.19
Crossing	Zebra	–	2.00	1.30
	Marked	–	2.00	1.30
	Island	–	2.00	1.40
	Traffic_signals	–	2.00	1.50

Algorithm 1. Generation of initial population**INPUT:** Graph G , Source node O , Destination node D , number of individuals ni **OUTPUT:** Initial population IP

```

1: Initialize  $IP$  as  $\emptyset$ 
2: Chose randomly one criterion  $c_i$  from  $\vec{c}$ 
3: Solve the SPP for  $c_i$ , obtain initial solution  $P_0$ 
4: Add  $P_0$  to  $IP$ 
5: while  $size(IP) < ni$  do
6:    $P_1 = Perturbation(P_0)$ 
7:   Add  $P_1$  to  $IP$ 
8:    $P_0 = P_1$ 
9: end while

```

is to minimize all the cost functions defined in Section 3.2. Therefore, a path P_1 with cost $\vec{c}_1 = (c_{1^1}, c_{1^2}, \dots, c_{1^k})$, dominates another path P_2 , with cost $\vec{c}_2 = (c_{2^1}, c_{2^2}, \dots, c_{2^k})$, if and only if $\forall i \in \{1 \dots k\}$, $c_{1^i} \leq c_{2^i}$ and $\exists i \in \{1 \dots k\}$, $c_{1^i} < c_{2^i}$. On the other hand, if two paths have exactly the same cost, they are designated as equivalent solutions. Solving the MOBRRP consists in finding a set of paths with enough diversity to fulfill the cyclists' preferences, and characterize the range of admissible values for each objective function.

4. MOBRRP approaches

Throughout this section, several details concerning the application of genetic algorithms to the MOBRRP will be addressed, namely, Subsection 4.1 presents genetic common points in the NGA-MOBRRP and in the NSGA-II implementation, while Subsection 4.2 presents the A* shortest path algorithm that was applied to generate the initial population, and to mutate existing solutions. Finally, Subsection 4.3 details the application of the NSGA-II in the MOBRRP context and Subsection 4.4 details the NGA-MOBRRP.

4.1. Genetic features applied to MOBRRP

4.1.1. Initial population

A new mechanism to generate the initial population of individuals is proposed. As shown in the pseudo-code of Algorithm 1, an initial solution P_0 is generated by randomly choosing one criterion and solving the SPP with the A* algorithm (Subsection 4.2). The initial solution is saved in a solutions' set, designated, initial population, IP , with a predefined size, and then, it is successively modified by the perturbation schema. All the generated solutions are added to the IP . This process was inspired by the first annealing stage of the MOSA proposed by Suppattatnarm et al. (2000), where all generated solutions are accepted and nondominated solutions are archived. The main difference is that in this approach, dominance tests are not performed, in order to speed up the process, thus, the IP set may contain dominated and nondominated solutions. Note that this approach differs from the one proposed by Nunes et al. (2020b), which used a multi-objective label-setting based algorithm to generate an initial population. Furthermore, this method is more efficient and

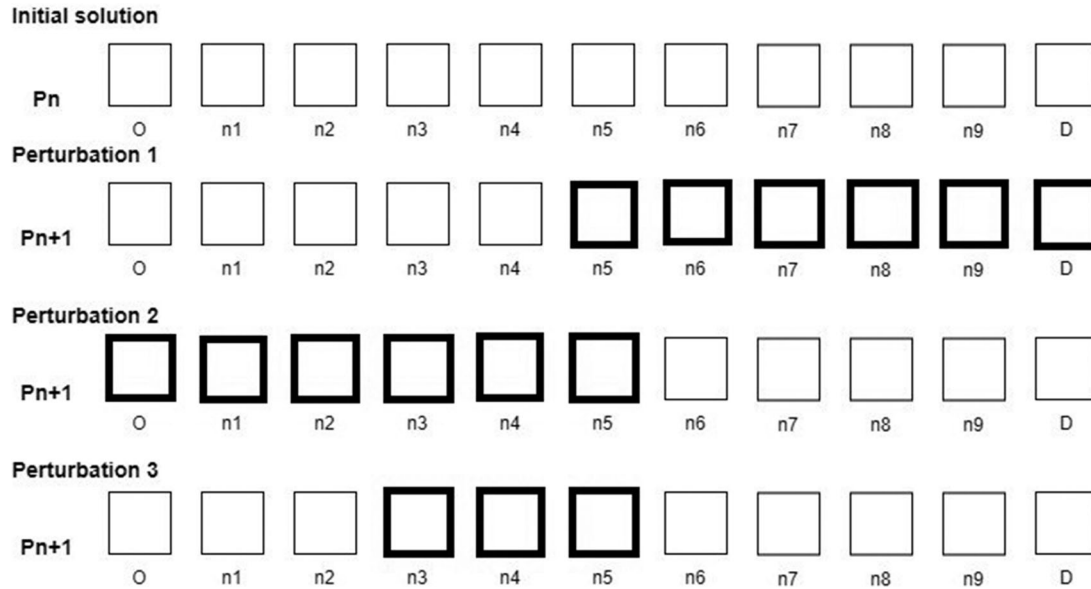


Fig. 1. Perturbation schema (Nunes et al., 2020a).

less sensitive in terms of computational performance, compared to the random walk and heuristic initialization proposed by Li et al. (2013), since a random walk in a real graph may take too long time to reach a path between the origin and destination nodes.

The function *Perturbation* consists in randomly change one solution, by applying one of the three perturbations described below and illustrated in Fig. 1.

- **Perturbation 1:** From a given solution P_n , a random node, n , is chosen. Then, a criterion c_i from $\vec{c}$ is randomly chosen. The sub-path between n and the destination, D is build by solving the SPP with the A* algorithm, according to c_i . The section of P_n , from n to D is replaced by the new sub-path.
- **Perturbation 2:** It is similar to perturbation 1, but instead of considering the section between n and D , it is considered the first section of P_n , that is, between the origin, O and n .
- **Perturbation 3:** Two random nodes from P_n , n_1 and n_2 are chosen, and the section between them is replaced by the shortest path, according to a randomly chosen criterion from $\vec{c}$.

4.1.2. Crossover

The crossover operator adopted to implement the NSGA-II and the NGA-MOBEP is the same proposed by Nunes et al. (2020b). Let $P1$ and $P2$ be two parent solutions. A common node to the two paths is randomly chosen, and the parents are combined in order to create two new paths, as represented in Fig. 2. If the two parent solutions do not have common nodes, except O and D , the crossover procedure returns the parent solutions. Despite the three genetic approaches, compared in this work, are elitists, the selection process for crossover and the population size control mechanisms are different in each approach and will be addressed in further subsections.

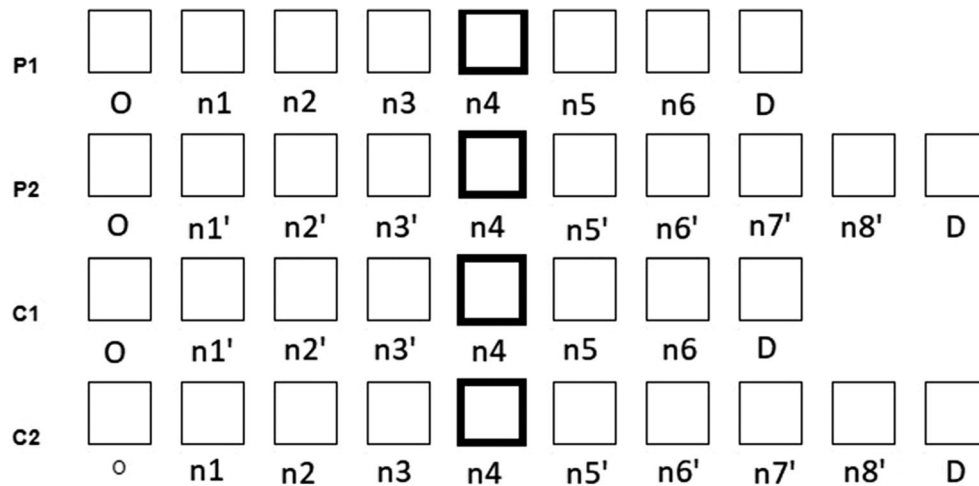


Fig. 2. Crossover (Nunes et al., 2020b).

4.1.3. Mutation

Mutation is a genetic operator that improves the algorithm's performance, by changing one or more nodes of one solution. It contributes for a wider exploration of the searching space and for the increasing of the population's diversity, which may lead to the achievement of good solutions that otherwise are not reached using the crossover operator alone. The mutation operator, proposed in this work, consists in applying one of the perturbations proposed by Nunes et al. (2020a) (described in Subsection 4.1.1) in a chosen solution.

4.2. A* shortest path algorithm

As mentioned, in some stages of the genetic approach, a mono-criterion shortest path algorithm is applied. It is mandatory that this algorithm has a good computational performance, since it is called several times, either to generate a population or to change an existing solution with the mutation operator. The Dijkstra's and uniform cost search (UCS) algorithms are very popular to solve the SPP. In this work, the A* algorithm was considered, since it is a variation of UCS, which uses a heuristic to estimate the cost between a node being visited and the arriving node. This procedure speeds up the search and if it is ensured that the estimation does not overestimate the total cost, the algorithm outputs the optimal path (LaValle, 2006).

The A* algorithm was implemented, using the NetworkX Python library, and its pseudo-code is represented in Algorithm 2. The search starts at the source node, O , which is assigned as the first *father_node*. Two lists are initialized, the *open_list* and the *closed_list*. The neighbors of the *father_node* are visited and the cost of going from O to each *neighbor_node*, g is calculated by adding the stored cost of transverse all edges to reach that node. The total cost to the destination, D , is estimated by f , $f = g + h$, where h is an estimation of the cost of going from *neighbor_node* to D . Each *neighbor_node* is added to the *open_list*, the *father_node* is removed from the *open_list* and added to the *closed_list*. Then the *open_list* is sorted in ascending order of f , and the next

Algorithm 2. A* shortest path algorithm**INPUT:** Graph G , Source node O , Destination node D , Criterion c **OUTPUT:** Shortest path P

```

1:  initialize closed_list as  $\emptyset$ 
2:  initialize open_list as  $\emptyset$ 
3:  initialize successor as  $\emptyset$ 
4:  add  $O$  to open_list
5:  initialize  $f(O) = 0$ 
6:  father_node =  $O$ 
7:  while father_node  $\neq D \wedge$  open_list  $\neq \emptyset$  do
8:    for neighbor_node  $\in$  father_neighbors do
9:      if neighbor_node  $\notin$  closed_list then
10:        successor(neighbor_node) = father_node
11:        calculate  $g(\text{neighbor\_node})$ 
12:        estimate  $h(\text{neighbor\_node})$  according to  $c$ 
13:         $f(\text{neighbor\_node}) = g + h$ 
14:      end if
15:      add father_node to closed_list
16:      remove father_node from the open_list
17:      sort open_list in ascending order of  $f$ 
18:      father_node = first element of open_list
19:    end for
20:  end while
21:  current_node =  $D$ 
22:   $P = \emptyset$ 
23:  add  $D$  to  $P_0$ 
24:  while current_node  $\neq O$  do
25:    Try:
26:      add successor(current_node) to  $P$ 
27:      current_node = successor(current_node)
28:    Except:
29:      print message “path not reachable”
30:  end while

```

father_node is the first node of *open_list*. The search continues, until the destination is reached and the *open_list* is not empty. Finally, the backtracking is made, that is, the nodes to be traversed are added to the path, from the destination to the source. Note that due to the street traffic rules, a path between origin and destination points may not exist. In these cases, an exception is reached, and a message is showed to the user.

To improve the computational performance and reach good solutions for the SPP, it is necessary to choose properly the heuristic functions for each criterion. A good and fast estimation of the distance between one node and the destination is the straight distance, because it is always lower than the real distance between the two nodes. Thus, the heuristic function for the distance criterion h_{dc} is given by the haversine distance between the *neighbor_node* and D (Equation 1). The heuristic for the travel time criterion, h_{ttc} is defined by Equation (10).

$$h_{ttc} = \frac{L}{V_{max}(sl)}, \quad (10)$$

where L is the straight distance (calculated by the haversine formula) between the *neighbor_node* and D , and $V_{max}(sl)$ is the maximum estimated speed for a road, according to Romero et al. (2015).

Comfort and safety criteria were defined as perceived distances as mentioned in Subsection 3.2. They may be shorter or longer than the real distances, depending on the road features. It is not possible to estimate the type of infrastructures between the *neighbor_node* and D , for this reason, the heuristic h was calculated according to Equation (11), where LB is the lower bound for the related features, according to each criterion.

$$h = L \times LB. \quad (11)$$

The mutation operator allows the exploration of new nodes, which were not in the previous population. Since the parts of the individual that are modified represent optimal paths according to one of the considered criteria, with an infinity number of iterations, the approach is capable to explore the full optimal pareto set. Moreover, the presented mutation operator is more efficient computationally than the one proposed by Li et al. (2013), which applied the Dijkstra's algorithm (slower, compared to the proposed A*), and also inserts more variability in the population, since three different perturbations are proposed.

4.3. NSGA-II approach

The pseudo-code of the NSGA-II strategy proposed by Deb et al. (2000) is represented in Algorithm 3. It is an elitist genetic algorithm with some special characteristics. It sorts the individuals in fronts F_i , according to nondomination levels, that is, the best front of solutions is F_0 , while other fronts represent solutions that are dominated at least by one solution in the population. In each front, the solutions are sorted in descending order of crowding distance value, which is a parameter based on a density estimator, whose aim is to maintain the diversity in the population.

This approach is widely used in multi-objective problems, because it is fast and usually achieves good approximated Pareto sets. However, with regard to the MOB RP, this approach has some weaknesses, namely:

- **Intrinsic randomness:** The selection method used in the original paper of Deb et al. (2000) is the binary tournament selection, where two random pairs of solutions are selected, and in each pair the best one is selected, based on the sorted ranking. Therefore, some of the best solutions in the population may not cross or mutate. Moreover, in the context of MOB RP, many similar paths may be generated, thus, to increase the diversity of solutions is important to ensure that the solutions that will be crossed are considerably different to produce new diversified solutions.
- **Population size:** The population size is fixed, and, in problems where the number of potential nondominated solutions is unknown, such as MOB RP, it may be lower than the number of nondominated solutions. Thus, in each generation, nondominated solutions could be discarded.
- **Lack of strategy to deal with repeated solutions:** When generating the initial population, due to the intrinsic randomness of the procedure, a population may have the same individual

Algorithm 3. NSGA-II template (adapted from (Deb et al., 2000))**INPUT:** Graph G , Source node O , Destination node D , number of individuals N , number of generations, NG **OUTPUT:** Approximate Pareto set PS

```

1:   Generate initial population,  $P_0$  of size  $N$ 
2:   Set  $t = 0$ 
3:   while  $t < NG$  do
4:     Calculate  $P_t$  fitness and sort  $P_t$  in fronts  $F_i$ 
5:     Calculate crowding distance in  $F_i$ 
6:     Sort  $F_i$  in descending order of crowding distance
7:     Generate offspring population  $Q_t$  of size  $N$ , using binary tournament selection, recombination and mutation operators
8:     Combine parent and offspring population  $R_t = P_t \cup Q_t$ 
9:     Sort  $R_t$  in fronts  $F_i$ 
10:    Initialize  $P_{t+1}$  as  $\emptyset$ 
11:     $i = 0$ 
12:    while  $\text{size}(P_{t+1}) < N$  do
13:      Calculate crowding distance in  $F_i$ 
14:       $P_{t+1} = P_{t+1} \cup F_i$ 
15:       $i = i + 1$ 
16:    end while
17:    Sort  $P_{t+1}$  in descending order of crowding distance
18:     $P_{t+1} = P_{t+1}[0 : N]$ 
19:     $t = t + 1$ 
20:  end while
21:  Initialize  $PS$ ,  $PS = \emptyset$ 
22:  Add nondominated solutions to  $PS$ ,  $PS = PS \cup F_0$ 

```

repeated several times. It degrades the algorithm performance and the population's diversity, since these solutions are seen as equivalent and are maintained in the same front. In some problems, we may have equivalent solutions that are very different, but most of the time, in the specific context of MOBSP, when this occurs, it means that the same solution was achieved multiple times.

4.4. NGA-MOBSP

The NGA-MOBSP is represented in Algorithm 4, and intends to solve the identified gaps of NSGA-II when applied to this problem, by implementing the following modifications:

- **Semi-controlled population size:** When inserting a front, F_i , in the new population, P_{t+1} , the full front is maintained, even if the defined population size, N , is exceeded. Thus, it is not necessary to calculate the crowding distance, and it is ensured that all nondominated solutions are preserved between generations.
- **Selection method:** The binary tournament selection does not ensure that all individuals are chosen for crossover. The proposed selection method chooses two different criteria from $\vec{c}$ and creates two ordered lists in ascending order of the respective cost. Then, the first element of each list is

Algorithm 4. Proposed genetic approach**INPUT:** Graph G , Source node O , Destination node D , number of individuals N , number of generations, NG **OUTPUT:** Approximate Pareto set PS

```

1:  Generate initial population,  $P_0$  of size  $N$ 
2:  Set  $t = 0$ 
3:  while  $t < NG$  do
4:    Calculate  $P_t$  fitness and sort  $P_t$  in fronts  $F_t$ 
5:    Generate offspring population  $Q_t$  of size equal to population size, using new selection method, recombination
      and mutation operators
6:    Combine parent and offspring population  $R_t = P_t \cup Q_t$ 
7:    Remove repeated solutions from  $R_t$ 
8:    Sort  $R_t$  in fronts  $F_t$ 
9:    Initialize  $P_{t+1}$  as  $\emptyset$ 
10:    $i = 0$ 
11:   while  $\text{size}(P_{t+1}) < N$  do
12:      $P_{t+1} = P_{t+1} \cup F_i$ 
13:      $i = i + 1$ 
14:   end while
15:    $t = t + 1$ 
16: end while
17: Initialize PS,  $PS = \emptyset$ 
18: Add nondominated solutions to PS,  $PS = PS \cup F_0$ 

```

selected to reproduce and is removed from both lists, ensuring that all individuals are only crossed once, increasing the population diversity.

- **Removing duplicated solutions:** In each generation, duplicated solutions are removed from the population.

Note that mutation and crossover operators require the concatenation of different paths, which theoretically may result in paths with loops. However, as the concatenated paths are optimal solutions for one of the criteria, and the costs are additive, this situation will never occur. Moreover, paths with loops would have a low score in the selection process (will be dominated by other solutions), thus will be naturally discarded by the NGA-MOBEP algorithm.

5. Experimental analysis

This section presents the results of computational experiments performed to test the effectiveness of the NGA-MOBEP, by comparing the quality of the obtained solutions and computational time with the well-known NSGA-II and with the works proposed by Nunes et al. (2020a, 2020b). Subsection 5.1 presents the computational setup and the parameters adopted for each algorithm, as well as the test instances. Subsection 5.2 details the quality indicators used to compare each algorithm, and finally, Subsection 6.1 presents the computational results considering a bi-objective problem. Subsection 6.2 presents the results considering the four objectives.

5.1. Experimental setup and test instances generation

The meta-heuristics tested may achieve different results in each run, thus, were performed 10 runs of each algorithm for each problem instance. The implementation was made in Python language and the experiments were made by using the Pypy 7.3.4 interpreter, in an Intel Core (TM) i7-10610U CPU, 1.80-2.30GHz machine.

The parameters for the MOSA algorithm were set as indicated by Nunes et al. (2020a), while the parameters for the genetic based algorithms, were set as follows:

- **Initial population:** In the genetic approach proposed by Nunes et al. (2020b), it was set at 500 individuals and the initial population in the NSGA-II and NGA-MOBGP was set to 50 individuals and this size was kept for population control.
- **Number of generations:** Number of times the genetic algorithm is performed. In the work presented by Nunes et al. (2020b), it was set at 22, while in the NSGA-II and in the NGA-MOBGP it was set at 60.
- **Mutation ratio:** Probability to occur the mutation of an individual. In Nunes et al. (2020b), a mutation operator was not employed. In the other approaches, a ratio of 0.1 was used in almost all experiments. However, a ratio of 0.05 was also tested for the NGA-MOBGP, for the four objectives scenario.

The computational tests were performed on two different graphs that represent real road networks of two cities in Portugal, namely Aveiro, with 9733 nodes and 21693 edges and Matosinhos, with 32,342 nodes and 71,302 edges. The maximum and average edge's length for the considered graphs are, respectively, 742.0 and 21.0 m. Eleven instances were generated, representing typically crowded places by tourists or students, such as universities, train stations, bus stops, shoppings, natural parks, hotels, or university residence. The instances A–E are the same presented by Nunes et al. (2020a), and were extracted from Aveiro's map, while instances F–K were extracted from Matosinhos' map.

In order to compute a lower bound to the solutions, the A* algorithm was applied for each criterion, and the real distances of each optimal path were calculated, and are represented in Table 3. The length's difference for each criterion reflects the existence of different road features that affect the cyclist's safety and comfort perception. The slope and the existence of obstacles are responsible for the difference between the shortest paths considering distance and travel time criteria. Note that the lengths of optimal paths' are similar in some instances, for example, in instances E and I, all optimal paths have similar length to the shorter path, which means that in these cases, the existing road features are such that the best solution for each criterion is similar to the shortest route.

5.2. Solution quality indicators

The effectiveness of techniques in multi-objective optimization is more complex than in single objective optimization, where the output can be compared with unique lower or upper bounds. In this paper, two different indicators to compare the algorithms were applied: the hypervolume indicator and the number of nondominated solutions identified, Zitzler et al. (2003).

Table 3
Length of the shortest paths computed by the A* algorithm (m)

	Distance	Time	Comfort	Safety
A	1883	2032	2571	2576
B	3340	3516	3887	4008
C	1009	1158	1187	1278
D	2171	2190	2218	2232
E	729	739	778	763
F	3969	4024	4127	4127
G	4765	4783	4963	5318
H	4689	4689	4689	5440
I	2845	2845	2845	3274
J	4382	4399	4382	4758
K	4765	4783	4963	5318

5.2.1. Hypervolume

This indicator is used to measure the quality of a set $S = \{p^{(1)}, p^{(2)}, \dots, p^{(j)}\}$ of j nondominated solutions computed by heuristic methods in multi-objective optimization problems. In the scope of this work, it is the measure of the region that is simultaneously dominated by S and bounded above by a reference point $r \in R^k$, such that $r \geq p^{(i)}$, $i \in \{1 \dots j\}$ and $p^{(i)} \in R^k$. The reference point is determined, by running the A* algorithm for each criterion and choosing the higher computed cost for each objective. Several algorithms have been proposed to calculate the hypervolume indicator in any number of dimensions. In this work, it is used a re-implementation of the code by Fonseca et al. (2006) in pure Python, which uses a recursive algorithm to compute this indicator for $k > 3$.

5.2.2. Number of non-dominated solutions

A solution may be nondominated in a specific set, but dominated by other solutions from a different one. Having this in consideration, after running all the algorithms, we build an approximated Pareto set, containing all nondominated solutions achieved by all algorithms. Let us refer to this approximation set as E^* . The number of solutions identified by each algorithm that belongs to E^* is used as quality indicator.

6. Results and discussion

The work proposed by Nunes et al. (2020b) has an initial population size that is considerably high, compared with these approaches. It occurs due to the different strategies to generate the initial population. Nunes et al. (2020b) employs a label-based search algorithm to generate a data structure named *Bag*, from which several paths could be achieved, while in this new approach the initial population is generated with the A* algorithm (Subsection 4.1.1), thus, the computational time depends on the population size. Furthermore, Nunes et al. (2020b) do not apply a mutation operator, thus the diversity of the achieved set is intrinsically dependent on the diversity of the initial

Table 4
Number of nondominated solutions (best and average)

Instance	GA		NGA-MOB RP		NSGA-II		MOSA	
	# Best	# Average	# Best	# Average	# Best	# Average	# Best	#Average
A	21	19.1	33	21.9	6	2.9	24	19.7
B	14	9.9	21	15.4	4	2.9	23	18.7
C	19	16.5	21	19.3	5	2.9	21	18.1
D	7	6.1	7	7.0	5	3.3	7	7.0
E	6	6.0	6	6.0	6	4.5	6	6.0
F	8	8.0	8	7.6	3	2.0	7	6.7
G	9	9.0	16	12.8	5	2.5	16	14.5
H	10	8.2	24	17.1	6	3.5	20	18.3
I	2	2.0	5	3.9	5	1.8	5	5.0
J	9	9.0	10	8.5	6	3.3	10	9.2
K	9	9.0	16	14.4	5	2.4	16	14.8

population. The number of generations is bigger in the NSGA-II and in the NGA-MOB RP, because the mutation operator increases the probability of improving solutions with the increasing number of generations.

6.1. Bi-criteria bike routing problem considering safety and travel distance

To have a fair comparison with the proposed strategies by Nunes et al. (2020a, 2020b), the four approaches are tested for the bi-objective problem (considering distance and safety criteria). Table 4 represents the number of nondominated solutions (highest value and average) for each instance. The algorithm proposed by Nunes et al. (2020b) is outperformed by the NGA-MOB RP in almost all instances. The exceptions are the instances D, E, F, and J, where both algorithms have similar performances. This is due to the fact that in Nunes et al. (2020b), the initial population is generated by a label-based algorithm, that is good to find solutions near the shortest path (considering the travel distance criterion), but has poorer performances to find paths considerably longer than this one. As can be seen in Table 4, the instances where it has good performance, compared to other approaches, are also the ones where the shortest paths computed by the A* algorithm, for each criterion, are similar. Furthermore, in this work, no mutation operator is applied, which is a factor that contributes to generate fewer solutions.

The NSGA-II has poor performances, due to the reasons mentioned in Subsection 4.3. Keeping repeated solutions in the population and the randomness in choosing the parents to reproduce, contribute to the lack of diversity, thus fewer nondominated solutions are identified.

The performances of the NGA-MOB RP and the MOSA algorithm are very similar. In general, the NGA-MOB RP identifies more nondominated solutions in the best running than MOSA. However, MOSA has the same average results for instances D and E, and even better average results for instances G–K and B. In general, MOSA is more consistent than the NGA-MOB RP. The randomness introduced by the mutation operator explains the difference between runs for the NGA-MOB RP.

Table 5
Hypervolume (average)

	GA	NGA-MOBSP	NSGA-II	MOSA
A	3.79E+05	3.95E+05	2.78E+05	3.93E+05
B	4.77E+05	5.52E+05	3.76E+05	5.83E+05
C	9.15E+04	9.30E+04	6.50E+04	9.26E+04
D	2.23E+04	2.23E+04	1.96E+04	2.23E+04
E	4.66E+03	4.66E+03	3.77E+03	4.66E+03
F	4.13E+04	4.13E+04	2.79E+04	4.12E+04
G	1.20E+05	1.53E+05	7.26E+04	1.58E+05
H	3.03E+05	3.09E+05	1.95E+05	3.11E+05
I	1.00E+02	1.81E+03	5.16E+02	2.58E+03
J	8.05E+04	8.47E+04	6.15E+04	8.83E+04
K	1.25E+05	1.56E+05	8.59E+04	1.59E+05

Table 6
Computational time (seconds) - Best

	GA	NGA-MOBSP	NSGA-II	MOSA
A	3.0	1.4	1.2	5.6
B	20.2	1.5	1.3	8.6
C	2.3	0.5	0.4	2.1
D	54.2	0.8	0.7	4.1
E	1.5	0.3	0.3	1.0
F	3.0	1.3	1.1	8.7
G	86.7	1.5	1.3	8.3
H	61.5	1.8	1.5	8.4
I	12.5	0.4	0.4	5.4
J	15.6	0.9	0.8	4.9
K	91.0	1.6	1.4	10.2

The results for the hypervolume indicator (average) are represented in Table 5. The results are compatible with the number of nondominated solutions, the algorithms with the best indicators are the MOSA and the NGA-MOBSP. This indicator is of utmost importance, since an algorithm may identify fewer solutions contained in E^* , but achieve several ones that are near to solutions belonging to E^* . In these cases, the hypervolume indicator may be greater for algorithms that identify fewer nondominated solutions.

The computational performance of the GA proposed by Nunes et al. (2020b) differs considerably in each instance, as can be seen in Table 6, it has a wide range of computational times. This occurs because the label-based search algorithm implemented to achieve the initial population is computationally heavy for instances with more nodes, and for those of which the computed paths are considerably longer than the shortest path, considering the travel distance criterion. The MOSA computational times also depend on the shortest paths' length and on the similarity between them. The longer the path, the slower is the A* algorithm (more nodes have to be visited to reach the optimal path) and each perturbation consumes more time. If the shortest paths for each criterion are similar, the MOSA convergence and the cooling schema are faster, which increases the

Table 7
Number of solutions (best and average)

Instance	GA		NGA-MOBRP		NSGA-II		MOSA	
	# Best	# Average	# Best	# Average	# Best	# Average	# Best	# Average
A	52	46.6	204	118.7	2	1.2	121	90.8
B	44	32.4	204	150.2	3	1.6	104	76.5
C	27	24.2	7	6.6	2	1.1	6	4.7
D	62	59.4	62	62.0	3	1.9	62	54.9
E	19	18.3	19	18.7	2	1.5	19	18.8
F	24	24.0	24	21.4	3	2.0	21	17.3
G	11	9.4	24	22.0	7	5.2	24	22.5
H	14	12.4	36	24.7	8	5.2	34	29.4
I	2	2.0	5	4.1	2	1.2	5	5.0
J	9	9.0	11	8.3	6	4.1	11	9.8
K	10	9.1	24	20.8	8	5.0	25	23.4

computational performance. The NGA-MOBRP and the NSGA-II have the best computational time performance. The computational time variation of the NSGA-II and the NGA-MOBRP for different instances is related to the computed shortest paths' length, since the generation of the initial population and mutations have to run the A* algorithm. However, the NGA-MOBRP is a little slower than NSGA-II, due to the mechanism that removes repeated individuals in each generation. Furthermore, the population size is semi-controlled, thus in some generations the NGA-MOBRP may have more individuals than NSGA-II.

6.2. Four-criteria bike routing problem

The second step of experimental analysis consists in repeating the computational experiments considering the four criteria. The number of solutions achieved by each algorithm is presented in Table 7. They are consistent with the results obtained for the bi-objective problem. The NGA-MOBRP has the best results, concerning the best run, for the majority of instances, and on average, MOSA performs better in instances G–K and E. Note that the NGA-MOBRP has a semi-controlled population size, which allows it to reach a set of solutions greater than the initial population size (50). Interesting to note that the GA proposed by Nunes et al. (2020b) has the best performance for instance C. It is explainable by the fact that the shortest paths for this instance have almost the same length (Table 3). It has good performance on instances E and J for the same reason, the algorithm used to generate the initial population is very effective to find nondominated solutions near the shortest path.

Note that for the four-criteria case, the number of solutions may include too many solutions to be presented to the cyclist. In these cases, the framework/application should request some inputs to the cyclist, for example, the criterion that most suits his needs, and the number of solutions to display (n_s). Then, the n_s best solutions, according to the chosen criterion, will be displayed to the cyclist. Since the MOBRP is very subjective, especially for the comfort and safety criteria, it is

Table 8
Hypervolume (average)

	GA	NGA-MOB RP	NSGA-II	MOSA
A	1.56E+10	2.11E+10	1.91E+09	2.25E+10
B	2.48E+10	2.72E+10	2.61E+09	4.44E+10
C	3.70E+08	1.94E+06	0.00E+00	1.75E+06
D	3.41E+07	3.42E+07	7.88E+06	3.38E+07
E	1.14E+08	1.14E+08	2.61E+07	1.14E+08
F	4.65E+08	4.63E+08	3.51E+08	4.59E+08
G	9.36E+09	1.09E+10	7.59E+09	1.09E+10
H	1.27E+10	1.34E+10	7.68E+09	1.38E+10
I	0.00E+00	3.83E+05	9.20E+04	4.97E+05
J	3.42E+09	3.38E+09	2.45E+09	3.42E+09
K	9.41E+09	1.07E+10	8.42E+09	1.09E+10

fundamental for the cyclist to have a set of solutions to choose the path that is more suitable for him/her.

It is interesting to note that for instances A and B, despite the number of solutions (best and average) obtained by the NGA-MOB RP is better compared to MOSA, the hypervolume indicator is better for MOSA, as can be seen in Table 8. This is due to the fact that MOSA may identify less nondominated solutions, but some dominated solutions achieved are near the non-dominated set, E^* . The other results are in accordance to the number of nondominated solutions indicator.

In comparison to the bi-objective problem, the computational time is slower for all the approaches. For the GA (Nunes et al., 2020b), it occurs because during the search process to achieve the initial population, an increased number of labels are maintained, thus the number of dominance tests increases considerably, increasing the computational time.

In the other algorithms, computational time increases with the number of criteria, because the computational time of the A* algorithm differs for each criterion. It occurs because the shortest paths have different lengths for each criterion as shown in Table 3, and A* computational time increases with the increasing of the path's length. Moreover, the cost estimation between the *neighbor_node* and *D* is much more accurate for the travel distance criterion, because the heuristics used for the other criteria underestimate more the total cost.

Furthermore, MOSA approach needs the convergence of four different temperatures, which also slows the process.

Despite the good quality indicators and computational performance achieved by the NGA-MOB RP, it is interesting to study the effect of the mutation ratio in the computational time and solutions quality. New experiments were made, changing the mutation ratio from 0.1 to 0.05. Table 10 summarizes the obtained results. There is a decrease of quality indicators as well as the decreasing of computational time, due to the fact that fewer mutations are computed. The computational time decreasing is more significant than the decreasing of quality indicators, for the majority of instances, as Table 11 shows. However, the computational times obtained with NGA-MOB RP, presented in Table 9 are fast enough to develop a real time tool for cyclists. Moreover, the decrease of quality indicators is considerable, thus, it is preferable to use the mutation ration as 0.1.

Table 9
Computational time (seconds) - Best

	GA	NGA-MOBSP	NSGA-II	MOSA
A	3.4	1.6	0.7	36.1
B	82.3	1.8	1.0	50.9
C	1.6	0.6	0.5	6.3
D	550.2	1.0	0.7	19.7
E	6.7	0.5	0.4	5.9
F	4867.7	1.6	1.5	7.2
G	174.8	1.9	1.2	9.4
H	143.4	2.0	1.1	8.6
I	21.6	0.4	0.9	4.2
J	22.7	1.1	0.8	5.4
K	188.31	2.0	1.1	9.3

Table 10
Performance of the NGA-MOBSP (Mutation ratio = 0.05)

Instance	# Best	# Average	Hypervolume	Time (s) - Best
A	121	65.8	1.95E+10	1.0
B	143	84.1	1.97E+10	1.3
C	7	5.2	1.80E+06	0.5
D	62	61.6	3.42E+07	0.8
E	19	17.5	1.14E+08	0.4
F	24	18.2	4.62E+08	1.2
G	22	17.9	1.02E+10	1.2
H	36	20.9	1.30E+10	1.4
I	5	2.6	1.91E+05	0.2
J	9	6.4	2.97E+09	0.8
K	20	15.7	9.71E+09	1.2

Table 11
Performance's comparison of NGA-MOBSP with different mutation ratios

Instance	Decreasing of indicators with the decreasing of the mutation ratio (%)			
	# Best	# Average	Hypervolume	Time - Best
A	−40.7	−44.6	−7.9	−37.5
B	−29.9	−44.0	−27.8	−27.8
C	0.0	−21.2	−7.4	−16.7
D	0.0	−0.6	0.0	−20.0
E	0.0	−6.4	0.0	−20.0
F	0.0	−15.0	−0.3	−25.0
G	−8.3	−18.6	−6.4	−36.8
H	0.0	−15.4	−3.6	−30.0
I	0.0	−36.6	−50.0	−50.0
J	−18.2	−22.9	−12.3	−27.3
K	−16.7	−24.5	−9.5	−40.0

7. Conclusions

In this work, a mutation operator and a new population generation schema for a genetic algorithm are proposed to solve the MOBSP. The population generation schema is based on the A* algorithm, which employs different heuristics for each criterion in the MOBSP. A NGA-MOBSP, with a semi-controlled population size and a mechanism to remove repeated solutions from a population, is also proposed and implemented. Furthermore, the well-known NSGA-II is implemented to solve the MOBSP and both approaches were compared with the works proposed by Nunes et al. (2020a, 2020b). These works were extended and tested with a set of real instances, considering the bi-objective (travel distance and safety) and the four-objective problems.

The results show that the **NGA-MOBSP is the most suitable to** be employed within a real time tool for cyclists (considering the bi-objective problem), since it is the only approach that achieves good quality metrics in a reasonable computational time. Furthermore, the proposed approach shows similar computational times when increasing the number of criteria to $k = 4$, showing consistent performances even in larger networks. The computational time is affected mainly by the distance between origin and destination points, performing identically in two networks with different sizes.

An interesting insight was to realize that the genetic approach proposed by Nunes et al. (2020b) is very effective in some instances (the ones in which the computed shortest paths for each criterion are similar), but the search algorithm is not able to reach solutions far from the shortest path. Also, the mutation operator proposed that is based in the perturbation schema presented by Nunes et al. (2020a) improves the solutions quality indicators, however, this operator is more time-consuming than the crossover operator and this is the reason that makes MOSA slower compared to the NGA-MOBSP.

Acknowledgments

The authors acknowledge the financial Support through projects, POCI-01-0247-FEDER-033769 – “Ghisallo - Investigação e Desenvolvimento de uma nova solução de comutação urbana, assente num novo conceito de veículo elétrico de próxima geração” that was funded by the Operational Program for Competiveness and Internationalization (COMPETE 2020) in its component FEDER and UIDB/EMS/00481 /2013-FCT CENTRO-01-0145 FEDER-022083 POCI-01- 0247-FEDER-033769; UIDB/00481/2020 and UIDP/00481/2020 - Fundação para a Ciência e a Tecnologia; and CENTRO-01-0145-FEDER-022083 - Centro Portugal Regional Operational Programme (Centro2020), under the PORTUGAL 2020 Partnership Agreement, through the European Regional Development Fund.

References

- Akar, G., Clifton, K., 2009. Influence of individual perceptions and bicycle infrastructure on decision to bike. *Transportation Research Record: Journal of the Transportation Research Board* 2140, 165–172.

- Arunadevi, J., Johnsanneekumar, A., Sujatha, N., 2007. Intelligent transport route planning using parallel genetic algorithms and MPI in high performance computing cluster. *15th International Conference on Advanced Computing and Communications (ADCOM 2007)*, IEEE, Piscataway, NJ, pp. 578–583.
- Bahmankhah, B., Fernandes, P., Coelho, M.C., 2019. Cycling at intersections: a multi-objective assessment for traffic, emissions and safety. *Transport* 34, 3, 225–236.
- Broach, J., Dill, J., Gliebe, J., 2012. Where do cyclists ride? A route choice model developed with revealed preference GPS data. *Transportation Research Part A: Policy and Practice* 46, 10, 1730–1740.
- Buehler, R., Dill, J., 2016. Bikeway networks: a review of effects on cycling. *Transport Reviews* 36, 1, 9–27.
- Chang, T.S., Nozick, L.K., Turnquist, M.A., 2005. Multiobjective path finding in stochastic dynamic networks, with application to routing hazardous materials shipments. *Transportation Science* 39, 3, 383–399.
- Chen, H.C., Wei, J.D., 2011. Using neural networks for evaluation in heuristic search algorithm. *Proceedings of the Twenty-Fifth AAAI Conference on Artificial Intelligence* 1, 1, 1768–1769.
- Deb, K., Agrawal, S., Pratap, A., Meyarivan, T., 2000. A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II. *PPSN 2000: Parallel Problem Solving from Nature PPSN VI* 1917, 849–858.
- Demir, E., Bektaş, T., Laporte, G., 2014. The bi-objective pollution-routing problem. *European Journal of Operational Research* 232, 3, 464–478.
- Dondi, G., Simone, A., Lantieri, C., Vignali, V., 2011. Bike lane design: the context sensitive approach. *Procedia Engineering* 21, 897–906.
- Duque, D., Lozano, L., Medaglia, A.L., 2015. An exact method for the biobjective shortest path problem for large-scale road networks. *European Journal of Operational Research* 242, 3, 788–797.
- ECF, 2019. The benefits of cycling. Technical Report, European Cyclists' Federation, Brussels.
- Ehrgott, M., Wang, J.Y., Raith, A., Van Houtte, C., 2012. A bi-objective cyclist route choice model. *Transportation Research Part A: Policy and Practice* 46, 4, 652–663.
- European Commission Mobility and Transport, 2019. Mobility and transport.
- Fonseca, C.M., Paquete, L., López-Ibáñez, M., 2006. An improved dimension-sweep algorithm for the hypervolume indicator. *2006 IEEE International Conference on Evolutionary Computation*, IEEE, Piscataway, NJ, pp. 1157–1163.
- Hochmair, H.H., Fu, Z., 2009. Web based bicycle trip planning for Broward County, Florida. *ESRI User Conference*, pp. 1–12.
- Hrncir, J., Song, Q., Zilecky, P., Nemet, M., Jakob, M., 2014. Bicycle route planning with route choice preferences. *Frontiers in Artificial Intelligence and Applications* 263, 1149–1154.
- Hrncir, J., Zilecky, P., Song, Q., Jakob, M., 2015. Speedups for multi-criteria urban bicycle routing. *OASIS - OpenAccess Series in Informatics* 48, 28.
- Hrncir, J., Zilecky, P., Song, Q., Jakob, M., 2017. Practical multicriteria urban bicycle routing. *IEEE Transactions on Intelligent Transportation Systems* 18, 3, 493–504.
- Jozefowicz, N., Semet, F., Talbi, E.G., 2008. Multi-objective vehicle routing problems. *European Journal of Operational Research* 189, 2, 293–309.
- Kang, L., Fricker, J.D., 2018. Bicycle-route choice model incorporating distance and perceived risk. *Journal of Urban Planning and Development* 144, 4, 04018041.
- Koning, M., Kopp, P., 2014. Are bicycles good for Paris? *International Journal of Transport Economics* 41, 3, 399–423.
- LaValle, S.M., 2006. *Planning Algorithms*, Vol. 9780521862. Cambridge University Press, Cambridge.
- Li, R., Leung, Y., Huang, B., Lin, H., 2013. A genetic algorithm for multiobjective dangerous goods route planning. *International Journal of Geographical Information Science* 27, 6, 1073–1089.
- Lindsay, G., Macmillan, A., Woodward, A., 2011. Moving urban trips from cars to bicycles: impact on health and emissions. *Australian and New Zealand Journal of Public Health* 35, 1, 54–60.
- Luxen, D., Gmbh, N., Vetter, C., 2011. Real-time routing with Openstreetmap data categories and subject descriptors. *Proceedings of the 19th ACM SIGSPATIAL GIS Conference*, pp. 513–516.
- Machuca, E., Mandow, L., 2012. Multiobjective heuristic search in road maps. *Expert Systems with Applications* 39, 7, 6435–6445.
- Martínez-Salazar, I.A., Molina, J., Ángel-Bello, F., Gómez, T., Caballero, R., 2014. Solving a bi-objective transportation location routing problem by metaheuristic algorithms. *European Journal of Operational Research* 234, 1, 25–36.
- Martins, E.Q.V., 1984. On a multicriteria shortest path problem. *European Journal of Operational Research* 16, 2, 236–245.

- Menghini, G., Carrasco, N., Schüssler, N., Axhausen, K.W., 2010. Route choice of cyclists in Zurich. *Transportation Research Part A: Policy and Practice* 44, 9, 754–765.
- Nunes, P., Moura, A., Santos, J., Completo, A., 2020a. A simulated annealing algorithm to solve the multi-objective bike routing problem. 2021 International Symposium on Computer Science and Intelligent Controls (ISCSIC), pp. 39–45.
- Nunes, P., Moura, A., Santos, J.P., 2020b. Evolutionary approach for the multi-objective bike routing problem. *Computational Logistics*. Springer, Berlin, pp. 311–325.
- Osaba, E., Del Ser, J., Bilbao, M.N., Lopez-Garcia, P., Nebro, A.J., 2018. Multi-objective design of time-constrained bike routes using bio-inspired meta-heuristics. In *Bioinspired Optimization Methods and their Optimizations*, Vol. 10835. Springer, Berlin, pp. 197–210.
- Pacheco, J., Caballero, R., Laguna, M., Molina, J., 2013. Bi-objective bus routing: an application to school buses in rural areas. *Transportation Science* 47, 3, 397–411.
- Paixão, J.M., Santos, J.L., 2013. Labeling methods for the general case of the multi-objective shortest path problem - a computational study. *Computational Intelligence and Decision Making* 61, 489–502.
- Pucher, J., Buehler, R., 2008. Making cycling irresistible: lessons from the Netherlands, Denmark and Germany. *Transport Reviews* 28, 4, 495–528.
- Raith, A., Ehrgott, M., 2009. A comparison of solution strategies for biobjective shortest path problems. *Computers and Operations Research* 36, 4, 1299–1331.
- Romero, J.P., Moura, J.L., Ibeas, A., Alonso, B., 2015. A simulation tool for bicycle sharing systems in multimodal networks. *Transportation Planning and Technology* 38, 6, 646–663.
- Sankararao, B., Yoo, C.K., 2011. Development of a robust multiobjective simulated annealing algorithm for solving multiobjective optimization problems. *Industrial and Engineering Chemistry Research* 50, 11, 6728–6742.
- Schmitt, J.P., Baldo, F., 2018. A method to suggest alternative routes based on analysis of automobiles' trajectories. 2018 XLIV Latin American Computer Conference (CLEI), IEEE, Piscataway, NJ, pp. 436–444.
- Song, Q., Zilecky, P., Jakob, M., Hrnčir, J., 2014. Exploring pareto routes in multi-criteria urban bicycle routing. 2014 17th IEEE International Conference on Intelligent Transportation Systems, ITSC 2014 1, 1781–1787.
- Stinson, M.A., Bhat, C.R., 2003. Commuter bicyclist route choice: analysis using a stated preference survey. *Transportation Research Record: Journal of the Transportation Research Board* 1828, 1, 107–115.
- Suppapatnarm, A., Sefen, K.A., Parks, G.T., Clarkson, P.J., 2000. A simulated annealing algorithm for multiobjective optimization. *Engineering Optimization* 33, 1, 59–85.
- Vedel, S.E., Jacobsen, J.B., Skov-Petersen, H., 2017. Bicyclists' preferences for route characteristics and crowding in Copenhagen - a choice experiment study of commuters. *Transportation Research Part A: Policy and Practice* 100, 53–64.
- Vidal, T., Crainic, T.G., Gendreau, M., Prins, C., 2014. A unified solution framework for multi-attribute vehicle routing problems. *European Journal of Operational Research* 234, 3, 658–673.
- Zitzler, E., Thiele, L., Laumanns, M., Fonseca, C., da Fonseca, V., 2003. Performance assessment of multiobjective optimizers: an analysis and review. *IEEE Transactions on Evolutionary Computation* 7, 2, 117–132.

Copyright of International Transactions in Operational Research is the property of Wiley-Blackwell and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.